

Neural Network from Scratch: A Deep Dive into Single Neuron Classification

👉 [Click here to view README.pdf](#) (./README.pdf).

Understanding Gradient Descent, Backpropagation, and the Math Behind Machine Learning, One Neuron at a Time

This project implements a complete neural network learning pipeline from scratch using only `numpy`. We train a single logistic neuron to classify handwritten digits (0 vs 1) from the MNIST dataset, providing a crystal-clear explanation of every mathematical concept: forward propagation, binary cross-entropy loss, the chain rule, backpropagation, and gradient descent.

Whether you're a beginner trying to understand how neural networks actually learn, or a practitioner who wants to see the math behind the code, this project will show you exactly how a single neuron learns from data.

What This App Does

The app downloads MNIST through `sklearn.datasets.fetch_openml` using `parser="liac-arff"`, keeps only labels `0` and `1`, normalizes pixel values to the range `[0, 1]`, trains one logistic neuron, prints train and test accuracy, and saves the loss curve to `artifacts/loss_curve.png`.

Each image has `28 x 28 = 784` pixels, so every sample is represented as a vector:

$$x = [x_1, x_2, x_3, \dots, x_{784}]$$

The model learns one weight for each pixel and one bias term:

$$w = [w_1, w_2, w_3, \dots, w_{784}]$$

$$b = \text{bias}$$

How The Single Neuron Learns

The single neuron learns by following a simple but powerful process: **make a prediction, measure the error, calculate how to improve, and update**. This is the essence of gradient descent. Let's walk through each step with the actual math formulas used in this

app.

Step 1: Forward Pass - Making Predictions

The neuron's job is to look at an image and output a probability that it's a (vs a).

1.1 Compute the Weighted Sum

The neuron first calculates a raw score by multiplying each pixel by its learned weight and adding a bias:

$$z = x \cdot w + b = \sum_{i=1}^{784} x_i w_i + b$$

What each term means:

- x_i = the brightness of pixel i (0 = black, 1 = white, after normalization)
- w_i = the weight the neuron has learned for pixel i (positive = makes z bigger, suggests digit ; negative = makes z smaller, suggests digit)
- b = bias term (shifts the decision boundary)
- z = the raw score before converting to probability

What does z tell us?

- If z is a **large positive number**, the neuron strongly believes the image is a
- If z is a **large negative number**, the neuron strongly believes the image is a
- If z is **near 0**, the neuron is uncertain

1.2 Convert to Probability (Sigmoid Activation)

The raw score z can be any real number, but we need a probability between 0 and 1. The sigmoid function transforms z into a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

So the final prediction (probability that the image is a) is:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-(x \cdot w + b)}}$$

Why sigmoid?

- It maps any real number to the range (0, 1)

- $\sigma(0) = 0.5$ (the decision boundary)
- As $z \rightarrow \infty$, $\sigma(z) \rightarrow 1$ (confident it's a)
- As $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$ (confident it's a)

How to interpret $\hat{y}$:

- $\hat{y} = 0.95$ means 95% confident it's a (only 5% chance it's a)
- $\hat{y} = 0.05$ means 5% confident it's a (95% confident it's a)
- $\hat{y} = 0.50$ means completely uncertain (could be either)

Step 2: Measuring Mistakes - The Loss Function

Now the neuron has made a prediction $\hat{y}$, but was it right? To know, we compare $\hat{y}$ to the true label y (which is either 0 or 1, since we only have digits and).

The Binary Cross-Entropy Loss

The app uses the **binary cross-entropy loss** to measure how wrong the prediction was:

$$L = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

What each term means:

- m = number of training samples in the batch
- y_i = true label for sample i (0 for digit , 1 for digit)
- $\hat{y}_i$ = predicted probability for sample i (between 0 and 1)
- $\log$ = natural logarithm

Why this loss function works:

Let's see what happens in different scenarios:

Case 1: True label is ($y = 1$) The loss simplifies to: $L = -\log(\hat{y})$

- If $\hat{y} = 0.9$ (correct, confident): $L = -\log(0.9) \approx 0.105$ (small loss ✓)
- If $\hat{y} = 0.5$ (correct, uncertain): $L = -\log(0.5) \approx 0.693$ (medium loss)
- If $\hat{y} = 0.1$ (wrong, confident): $L = -\log(0.1) \approx 2.303$ (large loss ✗)

Case 2: True label is ($y = 0$) The loss simplifies to: $L = -\log(1 - \hat{y})$

- If $\hat{y} = 0.1$ (correct, confident): $L = -\log(0.9) \approx 0.105$ (small loss ✓)

- If $\hat{y} = 0.5$ (correct, uncertain): $L = -\log(0.5) \approx 0.693$ (medium loss)
- If $\hat{y} = 0.9$ (wrong, confident): $L = -\log(0.1) \approx 2.303$ (large loss ×)

Key insight: The loss heavily penalizes confident wrong predictions. This forces the neuron to be careful about making bold predictions unless it's really sure.

Step 3: Learning from Mistakes, Computing Gradients

Now we know how wrong the prediction was (via the loss). But simply knowing we're wrong doesn't help us improve. We need to know **how to change** the weights w and bias b to make better predictions next time.

The key idea: Calculate how much each weight contributed to the error (the gradient), then adjust in the opposite direction.

3.1 Prediction Error (The Derivative at Output)

For logistic regression with sigmoid activation and cross-entropy loss, the derivative at the output layer has a beautiful simplification:

$$dz = \hat{y} - y$$

What does dz tell us?

- $dz > 0$: We predicted too high (thought it was 1 when it was actually 0)
- $dz < 0$: We predicted too low (thought it was 0 when it was actually 1)
- $dz = 0$: Perfect prediction! No error.

3.2 Weight Gradient (How Each Weight Should Change)

To know how each weight should change, we compute the gradient with respect to each weight:

$$dw = \frac{1}{m} X^T dz$$

Expanded for a single weight w_j :

$$dw_j = \frac{1}{m} \sum_{i=1}^m x_{ij} \cdot dz_i$$

What does dw tell us?

The gradient dw_j represents how much weight w_j contributed to the overall error:

- If dw_j is **positive**: Increasing w_j would increase the loss $\rightarrow$ we should **decrease** w_j
 - If dw_j is **negative**: Increasing w_j would decrease the loss $\rightarrow$ we should **increase** w_j
 - If dw_j is **zero**: This weight doesn't affect the loss $\rightarrow$ no change needed
-

Deep Dive: Why X^T (Transpose) Appears in $dw = \frac{1}{m}X^T dz$

Let me explain $dw = \frac{1}{m}X^T dz$ with a focus on why X^T (the **transpose of X**) appears.

1. Understand the dimensions

Suppose:

- We have m training examples (batch size).
- Each example has n input features (e.g., $n = 784$ pixels).
- X is an $m \times n$ matrix: each row is one example, each column is one feature.
- dz is a column vector of size $m \times 1$: the error ($\hat{y}_i - y_i$) for each example.

Example with tiny numbers: $m = 3$ examples, $n = 4$ features. Then X is 3×4 , dz is 3×1 .

2. What we want: dw

dw should be a column vector of size $n \times 1$ (one gradient per weight). For weight w_j (associated with feature j), the gradient is:

$$\frac{\partial L_{\text{total}}}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m dz_i \cdot X_{i,j}$$

where $X_{i,j}$ is the value of feature j in example i .

3. Matrix multiplication form

If we write this for all j (all weights) at once, we want:

$$dw = \frac{1}{m} \begin{bmatrix} \sum_i dz_i \cdot X_{i,1} \\ \sum_i dz_i \cdot X_{i,2} \\ \vdots \\ \sum_i dz_i \cdot X_{i,n} \end{bmatrix}$$

Notice each element is the **dot product** of the column vector dz (size m) with the column of X for that feature.

In matrix terms, this column-wise accumulation is exactly $X^T dz$.

- X^T has size $n \times m$.
- dz has size $m \times 1$.
- Therefore, $X^T dz$ has size $n \times 1$, matching the shape of dw .

So:

$$dw = \frac{1}{m} X^T dz$$

4. Why the transpose? intuitive reason

Let's break down the **manual computation** step by step, using the same small example.

The setup

We have:

- $m = 2$ examples
- $n = 3$ features per example

Matrix X (examples in rows, features in columns):

$$X = \begin{bmatrix} 1 & 4 & 2 \\ 3 & 5 & 6 \end{bmatrix}$$

Error vector dz (one error per example):

$$dz = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}$$

We want dw the gradient for each weight (one per feature).

Manual formula for one weight

For weight w_j (associated with feature j):

$$dw_j = \frac{1}{m} \sum_{i=1}^m dz_i \cdot X_{i,j}$$

That is: multiply each example's error by its feature value, sum over all examples, then divide by m .

Compute dw_1 (first feature, column 1 of X)

- Example 1: $dz_1 = 0.2, X_{1,1} = 1 \rightarrow \text{contribution} = 0.2 \times 1 = 0.2$
- Example 2: $dz_2 = -0.1, X_{2,1} = 3 \rightarrow \text{contribution} = (-0.1) \times 3 = -0.3$

- Sum = $0.2 + (-0.3) = -0.1$
- Divide by $m = 2$: $-0.1/2 = -0.05$

Thus $dw_1 = -0.05$.

Compute dw_2 (second feature, column 2)

- Example 1: $0.2 \times 4 = 0.8$
- Example 2: $(-0.1) \times 5 = -0.5$
- Sum = $0.8 - 0.5 = 0.3$
- Divide by 2: $0.3/2 = 0.15$

$dw_2 = 0.15$.

Compute dw_3 (third feature, column 3)

- Example 1: $0.2 \times 2 = 0.4$
- Example 2: $(-0.1) \times 6 = -0.6$
- Sum = $0.4 - 0.6 = -0.2$
- Divide by 2: $-0.2/2 = -0.1$

$dw_3 = -0.1$.

$$\text{So } dw = \begin{bmatrix} -0.05 \\ 0.15 \\ -0.1 \end{bmatrix}.$$

Now do the same using $X^T dz$ (vectorised)

First, compute X^T (transpose of X):

$$X^T = \begin{bmatrix} 1 & 3 \\ 4 & 5 \\ 2 & 6 \end{bmatrix}$$

Now multiply X^T (size 3×2) by dz (size 2×1):

$$X^T dz = \begin{bmatrix} 1 \cdot 0.2 & + & 3 \cdot (-0.1) \\ 4 \cdot 0.2 & + & 5 \cdot (-0.1) \\ 2 \cdot 0.2 & + & 6 \cdot (-0.1) \end{bmatrix} = \begin{bmatrix} 0.2 - 0.3 \\ 0.8 - 0.5 \\ 0.4 - 0.6 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 0.3 \\ -0.2 \end{bmatrix}$$

Then divide by $m = 2$:

$$\frac{1}{2} X^T dz = \begin{bmatrix} -0.1/2 \\ 0.3/2 \\ -0.2/2 \end{bmatrix} = \begin{bmatrix} -0.05 \\ 0.15 \\ -0.1 \end{bmatrix}$$

Exactly the same as the manual result.

Why does this work?

Because $X^T dz$ collects, for each feature, the sum of $dz_i \times X_{i,j}$ over all examples which is exactly what we need before dividing by m . The transpose flips the matrix so that features become rows, allowing a clean matrix multiplication that performs the weighted sum in one step.

Deep Dive: The Backward Pass and Chain Rule

Let's break down the **backward pass** step by step. The name "backward pass" means we propagate the error from the output back to the parameters (weights and bias) to compute how much each contributed to the loss. This is done using the **chain rule** of calculus.

1. What We Have

- **Forward pass** gave us:

- $z = \mathbf{w} \cdot \mathbf{x} + b$
- $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$

- **Loss** for one example (binary cross-entropy):

- $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
- For many examples (batch of size m), the total loss is the average:
- $L_{\text{total}} = \frac{1}{m} \sum_{i=1}^m L_i$

Our goal: find how L_{total} changes when we change each weight w_j and the bias b – i.e., the partial derivatives $\frac{\partial L_{\text{total}}}{\partial w_j}$ and $\frac{\partial L_{\text{total}}}{\partial b}$.

2. Chain Rule: From Loss Back to Weights

We know the loss depends on $\hat{y}$, which depends on z , which depends on w_j and b . So:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

Similarly for b : $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial b}$

Let's compute each piece.

- a. $\frac{\partial L}{\partial \hat{y}}$ (derivative of loss w.r.t predicted probability)

For a single example:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

We can rewrite it as:

$$\frac{\partial L}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$$

b. $\frac{\partial \hat{y}}{\partial z}$ (derivative of sigmoid)

The derivative of the sigmoid is $\hat{y}(1 - \hat{y})$.

c. $\frac{\partial z}{\partial w_j}$ and $\frac{\partial z}{\partial b}$

From $z = w_1x_1 + \dots + w_jx_j + \dots + b$:

$$\frac{\partial z}{\partial w_j} = x_j, \quad \frac{\partial z}{\partial b} = 1$$

d. Putting it together

$$\frac{\partial L}{\partial w_j} = \left[\frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \right] \cdot [\hat{y}(1 - \hat{y})] \cdot x_j = (\hat{y} - y) \cdot x_j$$

The $\hat{y}(1 - \hat{y})$ cancels nicely! So for one example:

$$\frac{\partial L}{\partial w_j} = (\hat{y} - y) x_j$$

$$\frac{\partial L}{\partial b} = (\hat{y} - y) \cdot 1 = \hat{y} - y$$

3. Over a batch of m examples

To get the gradient of the **average loss** L_{total} :

$$\frac{\partial L_{\text{total}}}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{i,j}$$

$$\frac{\partial L_{\text{total}}}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)$$

4. Vectorised form

Let:

- X be an $m \times n$ matrix (each row is one example, n = number of features, e.g., 784 pixels)

- $\mathbf{y}$ be a column vector of true labels (size $m \times 1$)
- $\hat{\mathbf{y}}$ be the predicted probabilities (size $m \times 1$)
- $d\mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ (size $m \times 1$)

Then:

$$d\mathbf{w} = \frac{1}{m} \mathbf{X}^T d\mathbf{z} \quad (\text{size } n \times 1)$$

$$db = \frac{1}{m} \sum_{i=1}^m dz_i = \frac{1}{m} \cdot \text{sum}(d\mathbf{z})$$

5. Intuition behind $d\mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$

Notice that the prediction error $d\mathbf{z}$ is the simple difference between predicted probability and true label.

- If the true label is 1 ($y = 1$) and neuron predicts $\hat{y} = 0.9$, then $d\mathbf{z} = 0.9 - 1 = -0.1$ (negative).
- This negative value, multiplied by the input x_j and learning rate, will **decrease** the weight if x_j is positive – which is correct because the prediction was too high, so we want to lower it.

The simple formula $d\mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ emerges directly from the magical cancellation of sigmoid derivatives with binary cross-entropy – this is why the logistic neuron is so elegant.

6. Summary of the backward pass

1. Compute $d\mathbf{z} = \hat{\mathbf{y}} - \mathbf{y}$ (a vector of errors on the batch).
2. Compute $d\mathbf{w} = \frac{1}{m} \mathbf{X}^T d\mathbf{z}$ – each weight gets the average contribution from all examples (error $\times$ its input feature).
3. Compute $db = \frac{1}{m} \sum d\mathbf{z}$ – the bias gets the average error.

Then use these gradients to update $\mathbf{w}$ and $\mathbf{b}$ with gradient descent. The backward pass is the "learning" part – it tells each weight and bias how much to change to reduce the loss.

Step 4: Improving the Model - Gradient Descent Update

Now we know how to adjust each parameter. The gradient descent algorithm updates the parameters to reduce the loss:

Weight update:

$$w \leftarrow w - \alpha \cdot dw$$

Bias update:

$$b \leftarrow b - \alpha \cdot db$$

Where α is the **learning rate** (controlled by the `--lr` argument, default is 0.1).

Why do we subtract the gradient?

- The gradient `dw` points in the direction of **steepest increase** in loss
- We want to **decrease** the loss, so we move in the **opposite direction**
- The learning rate α controls the step size

What happens during training?

Scenario	What Happens	Learning Rate Effect
dw is large, x_j was bright	Weight gets updated a lot	Large $\alpha \rightarrow$ faster learning but might overshoot
dw is small, prediction was close	Weight gets updated a little	Small $\alpha \rightarrow$ slower but more stable learning
dz is positive (predicted too high)	Weight decreases	Optimal $\alpha \rightarrow$ converges smoothly to good weights

The Complete Learning Loop

Putting it all together, here's what happens in each training epoch:

1. Forward Pass: Make predictions

- Compute $z = Xw + b$ for all training samples (matrix multiplication)
- Apply sigmoid: $\hat{y} = \sigma(z) = 1/(1 + e^{-z})$

2. Loss Calculation: Measure how wrong we are

- Compute `L` using binary cross-entropy
- $L = -(1/m) * \sum(y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y}))$

3. Backward Pass: Calculate how to improve

- Compute prediction error: $dz = \hat{y} - y$

- Compute weight gradients: $dw = (1/m) * X^T * dz$
- Compute bias gradient: $db = (1/m) * \text{sum}(dz)$

4. **Parameter Update:** Apply the improvements

- Update weights: $w = w - \alpha * dw$
- Update bias: $b = b - \alpha * db$

5. **Progress Tracking:** Store loss for visualization

Repeat for many epochs (default is 100) until the loss stops decreasing and accuracy plateaus.

Why This Works: Key Insights

1. Weight Initialization Matters

We initialize weights from a normal distribution:

$$w \sim \mathcal{N}(0, 0.01)$$

Why small weights?

- Large initial weights → large z values → sigmoid saturates (gradient becomes nearly zero) → can't learn
- Small initial weights → moderate z values → sigmoid in active region (gradient ≈ 0.25) → can learn

2. Sigmoid Properties

The sigmoid function has important characteristics:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- Maps any real number to (0, 1) range
- $\sigma(0) = 0.5$ (the decision boundary)
- As $z \rightarrow \infty$, $\sigma(z) \rightarrow 1$ (confident it's a 1)
- As $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$ (confident it's a 0)

3. Loss Function Choice

Binary cross-entropy is ideal for binary classification:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Why it works:

- Penalizes confident wrong predictions heavily
- Works well with sigmoid (combined gradient simplifies to $dz = \hat{y} - y$)
- Convex loss landscape → guaranteed convergence to global minimum

4. Gradient Descent Dynamics

The update rule:

$$w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$$

How learning progresses:

- Early epochs: Large gradients → big weight updates → loss drops quickly
- Later epochs: Small gradients → fine-tuning → loss converges
- With proper learning rate: Smooth convergence to optimal weights

Decision Rule for Classification

After training, the neuron classifies images using:

$$\text{prediction} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases}$$

The threshold of 0.5 is the natural decision boundary for the sigmoid function since $\sigma(0) = 0.5$.

Setup

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Run

```
python3 single_neuron_mnist.py
```

Optional arguments:

```
python3 single_neuron_mnist.py --epochs 150 --lr 0.05
python3 single_neuron_mnist.py --no-plot
```

This training step now also saves:

- `artifacts/neuron_weights.npy`
- `artifacts/neuron_bias.npy`

Streamlit App

After training, launch the browser UI:

```
streamlit run streamlit_app.py
```

The app lets you draw a digit in the browser, preprocesses the canvas to `28x28`, and shows the predicted class (`0` or `1`) plus the probability of being `1`.

Expected Result

When training runs correctly, you should see:

- Loss values decreasing over epochs (from ~ 0.7 down to ~ 0.1 or lower)
- High training accuracy (typically $>95\%$)
- High test accuracy (typically $>95\%$)
- Saved model files in `artifacts/neuron_weights.npy` and `artifacts/neuron_bias.npy`
- A saved plot in `artifacts/loss_curve.png` showing the loss curve

Additional Notes

Binary Cross-Entropy (BCE)

Binary cross-entropy (BCE) is a loss function used in binary classification models (e.g., logistic regression, neural networks) where the output is a probability between 0 and 1. It measures how well the predicted probabilities match the true binary labels (0 or 1). The lower the BCE, the better the model's predictions.

Formula

For a single prediction $\hat{y}$ (predicted probability) and true label $y \in \{0, 1\}$:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Over a dataset of N samples, the average loss is:

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Intuition

- If the true label is $y = 1$, the loss becomes $-\log(\hat{y})$. This penalizes predictions $\hat{y}$ far from 1:
 - When $\hat{y} \rightarrow 1$, $\log(1) = 0 \rightarrow$ loss small.
 - When $\hat{y} \rightarrow 0$, $\log(0) \rightarrow -\infty \rightarrow$ loss huge.
- If $y = 0$, the loss becomes $-\log(1 - \hat{y})$. This penalizes predictions far from 0.

Essentially, BCE is the negative log-likelihood of a Bernoulli distribution. Minimizing it encourages the model to output high probabilities for the correct class.

Example

Suppose true label $y = 1$ and model predicts $\hat{y} = 0.9$:

$$L = -[1 \cdot \log(0.9) + 0 \cdot \log(0.1)] = -\log(0.9) \approx 0.105$$

If $\hat{y} = 0.6$: $L \approx 0.511$ (worse).

If $\hat{y} = 0.1$: $L \approx 2.303$ (very bad).

Why not mean squared error?

BCE works better than MSE for classification because it has a smoother, convex error surface when paired with a sigmoid output, leading to faster and more reliable gradient descent.

Gradient Descent

Gradient descent is an iterative optimization algorithm used to find the **minimum** of a function. In machine learning, it's the workhorse for training models: you use it to minimize the **loss function** (e.g., binary cross-entropy) by adjusting the model's parameters (weights).

Intuition

Imagine standing on a hill at night and wanting to reach the lowest valley. You can't see far, but you can feel the slope under your feet. Gradient descent says: *take a step in the steepest downhill direction*. The "slope" is the **gradient** (partial derivatives) of the loss with respect to each parameter.

Gradient Descent Dynamics

- **Gradient** points in the direction of *steepest increase* of the loss.
- So you move **opposite** the gradient to decrease the loss.

Mathematical update rule

For a parameter θ (e.g., a weight), the update at step t is:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t)$$

- η (eta) = **learning rate** – size of each step.
- $\nabla_{\theta} L$ = gradient of loss L with respect to θ .

How it works (step-by-step)

1. **Initialize** parameters randomly (or with zeros in some cases, but random is common).
2. **Compute** the loss on the current batch/data (e.g., BCE).
3. **Calculate** the gradient of the loss w.r.t. each parameter (using backpropagation in neural networks).
4. **Update** each parameter by subtracting a fraction (the learning rate) of the gradient.
5. **Repeat** until the loss stops decreasing (convergence) or after a fixed number of iterations.

Relation to binary cross-entropy

When you train a binary classifier with BCE as the loss, gradient descent tells you **how to change the weights** so that the predicted probabilities move closer to the true labels. The gradient of BCE with respect to the pre-activation output (logit) is simply $(\hat{y} - y)$, which is intuitive and well-behaved.

Variants (quick mention)

- **Batch GD** – uses entire dataset per update.
- **Stochastic GD (SGD)** – uses one random sample per update (faster, noisy).
- **Mini-batch GD** – uses a small batch (common in practice).
- **Adam, RMSprop**, etc. – adaptive learning-rate methods built on gradient descent.

Key takeaway

Gradient descent is the engine that drives learning: it repeatedly takes small, informed steps downhill on the loss surface until it (hopefully) reaches the bottom – the set of parameters that minimizes the loss.